École Polytechnique de l'Université de Tours
64, Avenue Jean
Portalis 37200
TOURS, FRANCE
(33)2-47-36-14-14
www.polytech.univ-tours.fr

Parcours des Écoles d'Ingénieurs Polytech

Année 2024 / année 2025

Projet : 'Chef de Gare'

Étudiants

Ivan EPOV
ivan.epov@etu.univ-tours.fr
Elias HAMDOUNI
elias.hamdouni@etu.univ-tours.fr

Encadrant

Yannick KERGOSIEN
yannick.kergosien@univ-tours.fr

-2 Avertissement

Ce document a été rédigé par ***Elias Hamdouni*** et ***Ivan Epov*** susnommés les auteurs. L'École Polytechnique de l'Université François Rabelais de Tours est représentée par ***Yannick Kergosien*** susnommé le tuteur académique.

Par l'utilisation de ce modèle de document, l'ensemble des intervenants du projet acceptent les conditions définies ci-après.

Les auteurs reconnaissent assumer l'entière responsabilité du contenu du document ainsi que toutes suites judiciaires qui pourraient en découler du fait du non-respect des lois ou des droits d'auteur.

Les auteurs attestent que les propos du document sont sincères et assument l'entière responsabilité de la véracité des propos.

Les auteurs attestent ne pas s'approprier le travail d'autrui et que le document ne contient aucun plagiat.

Les auteurs attestent que le document ne contient aucun propos diffamatoire ou condamnable devant la loi.

Les auteurs reconnaissent qu'ils ne peuvent diffuser ce document en partie ou en intégralité

sous quelque forme que ce soit sans l'accord préalable du tuteur académique et de l'entreprise.

Les auteurs autorisent l'école polytechnique de l'université François Rabelais de Tours à diffuser tout ou partie de ce document, sous quelque forme que ce soit, y compris après transformation en citant la source. Cette diffusion devra se faire gracieusement et être accompagnée du présent avertissement.

1 Table des matières

Chapitre 0 : Introduction……..……4
- 0.1 Présentation du projet 4
- 0.2 Présentation des objectifs fixés 4
- 0.3 Présentation de la technologie sélectionnée 4
- 0.4 Gestion du projet 5

Chapitre 1 : Présentation de Godot……..……..……..6
- 1.1 Arborescence et nœuds 6
- 1.2 Scènes 7
- 1.3 Scripts 8
- 1.4 Quelques nœuds et propriétés utilisés 8
  - 1.4.1 Métadonnées 8
  - 1.4.2 Path2D & PathFoloow2D 8
  - 1.4.3 Area2D & ColisionShape2D 8

Chapitre 2 : Game design……..……9
- 2.1 Aspects généraux du jeu 9
  - 2.1.1 But du jeu et score 9
  - 2.1.2 Actions du joueur 9
  - 2.1.3 Vue de l'utilisateur 10
- 2.2 Aspects spécifiques du jeu 11
  - 2.2.1 Apparition des trains 11
  - 2.2.2 Type de carte 11
  - 2.2.3 Difficulté 11

Chapitre 3 : Développement du jeu……..……12
- 3.1 Aspects généraux du jeu 12
  - 3.1.1 Création de la carte 12
  - 3.1.2 Chemin d'un train 13
  - 3.1.3 Détection de collisions et cliques du joueur 13
- 3.2 Aspects spécifiques du jeu 14
  - 3.2.1 Génération aléatoire de train 14

Chapitre 4 : Projet fini……..……15
Chapitre 5 : Conclusion……..……16
Chapitre 6 : Bibliographie……..……..17
Chapitre 7 : Comptes Rendus Hebdomadaires……..……18

0 Introduction

Ce chapitre introduit notre projet, nos objectifs et nos moyens utilisés pour réaliser 'Chef de Gare'.

Notre projet consiste en la création d'un jeu vidéo prénommé « Chef de Gare », où le joueur incarne un responsable ferroviaire chargé de gérer l'arrivée et le départ de différents trains.

0.1 Présentation du projet

« Chef de Gare » est un jeu de gestion de gare. Le joueur doit gérer l'arrivée et le départ de trains en les aiguillant sur les bonnes voies tout en évitant d'éventuels conflits et collisions entre ces trains. Chaque train a une destination bien précise que le joueur doit prendre en considération dans ces décisions de jeu.

0.2 Présentation des objectifs fixés

Afin de mener notre projet à bien, nous nous sommes fixés différents objectifs :

- Dans un premier temps, créer un jeu jouable et avec toutes les fonctionnalités que l'on voulait intégrer. En délaissant les graphismes : les différents éléments du jeu sont représentés de manière simplifiée par des formes géométriques et des couleurs.
- Puis, faire les graphismes afin de rendre le jeu plus esthétique dans son ensemble.
- Crée différents niveau de difficultés,
- Et, potentiellement, crée un algorithme de résolution automatique

0.3 Présentation de la technologie sélectionnée

Afin de réaliser ce projet, nous avons eu le choix entre 2 technologies : Python ou Godot.

La première, Python, est un langage de programmation interprété, très documenté et disposant de nombreuses librairies, certaine permettant notamment la réalisation de jeu vidéo (comme Pygame pour les jeux en 2D ou panda3D pour ce en 3 dimensions). Mais aucune de ces librairies ne contient d'IDE (Environnement de Développement Intégré - en anglais : Integrated Development Environment) dédié à la création de jeu (permettant de placer des objets par glisser déposer et d'avoir une visualisation globale de la scène de jeu par exemple), Tout doit se faire à l'aide de

code, y compris le placement des objets sur la scène de jeu, ce qui peut rendre la réalisation d'un projet important assez pénible et rends les scripts très denses. Python est une technologie plutôt globale, qui permet de tout faire, mais pas forcément de manière très optimisée. La création de l'algorithme de résolution automatique reste plus simple sur Python

La deuxième technologie, Godot, est un moteur de jeu libre de droits, open source et gratuit. Il est assez jeune mais dispose déjà d'un communauté importante, permettant d'avoir réponse à des problèmes rapidement. Il dispose d'un IDE complet : prenant en charge la gestion des différents objets du jeux, la création de script, la gestion de la scène de jeu etc. (à la manière d'Unreal Engine ou Unity). De plus, Godot propose différents objets intégrés simplifiant la création de nos propres objets tel que des boutons, des zones de détections, des images, des courbes… Ainsi, il simplifie grandement la gestion globale d'un projet de jeu vidéo comme le notre. Ce moteur utilise comme langage le GDScript, un langage très similaire à Python permettant une prise en main rapide. C'est donc une technologie spécialisée dans la réalisations de jeux vidéos, ce qui rend l'objectif de l'algorithme de résolution automatique difficile.

Nous avons choisie Godot pour la réalisation de « Chef de Gare » ; pour sa simplicité de prise en main et sa spécialisation dans le domaine du jeu vidéo.

Notre encadrant, Yannick Kergosien, ne connaît pas Godot : il n'a donc pas pu nous aider sur l'aspect développement du projet.

0.4 Gestion du projet

Nous avons avancé sur le projet de manière simultanée sur deux ordinateurs, respectivement nos ordinateurs personnels. La version principale était celle de Ivan Epov, tandis que celle de Elias Hamdouni était dédiée à la recherche d'idée et la résolution de problèmes, version régulièrement mise à jour.
Un cahier des charges nous a permis de ne pas s'éloigner de nos objectifs.

Voici la répartition des tâches :

Epov Ivan :
- réalisation du graphe simulant un complexe ferroviaire,
- implémentation des trains,
- algorithme de génération de train.

Hamdouni Elias :
- gestion des entrées utilisateurs (cliques),
- affichage du score
- apparence des trains et rails,
- création de niveaux,

1 Présentation de Godot

Ce Chapitre présente les bases du moteur de jeu utilisé dans notre projet : Godot.

Godot est un moteur de jeu fonctionnant à la manière d'autre grand moteur de jeu comme Unreal Engine et Unity. Il est open source et très simple de prise en main. Sa simplicité n'ampute pas sa polyvalence car ce dernier permet permet d'exporter un projet aussi bien sur PC (Windows Desktop, macOS, Linux) que sur mobile (iOS et Android) mais aussi en format web (HTML 5) pour les moins gourmands.
Il fonctionne grâce à un système d'arborescence et de nœuds.

1.1 Arborescence et nœud:

Dans Godot, la création des différents objets de notre projet se fait grâce à des nœuds. Un nœud est un élément d'un arbre, relié à d'autres nœuds de manière hiérarchisée. Il existe une grande de variété de nœud différents dans Godot ayant chacun des attributs communs ou particuliers.
Par exemple Node2D est un nœud classique contenant des attributs lui permettant d'être manipulé dans un espace à deux dimensions : position (ses coordonnées), scale (son échelle) ou encore rotation (son angle d'inclinaison)... Il en a évidement bien d'autres.
Deuxième exemple, Sprite2D. Ce dernier est héréditaire du nœud Node2D, il a donc toute ses attributs, mais contient en plus des attributs liées à l'affichage d'une image. Ainsi, par exemple, l'attribut 'texture' permet de contenir une image.

Tout ces différents nœuds son relié entre à la manière d'un arbre. Un nœud à donc un nœud parent (au dessus de lui) et un nœud enfant (en dessous de lui). Ce système crée tout un système de priorité. Les nœuds parents son prioritaires sur les nœuds enfant que ce soit dans l'affichage lors de l'exécution du jeu, que dans la modification des nœuds.
Un nœud parent sera donc affiché avant ses nœuds enfants sur l'écran, même si cela se compte en millisecondes.
Si un nœud parent est modifié, ses nœuds enfants seront impactés

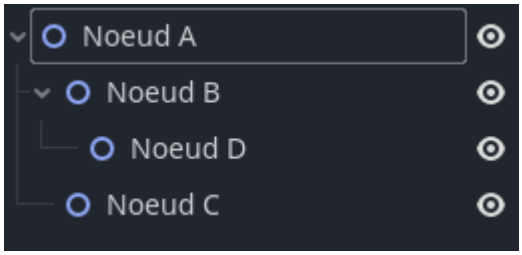


Figure 1.1.1 : Exemple d'un arbre simple

Figure 1.1.1 :

- Dans cet exemple, 'Noeud A' est le nœud parent de tout les autres nœuds, c'est le nœud à l'origine de l'arbre.
- Les nœuds B et C sont enfants du nœud A, à noter que le nœud B sera prioritaire sur le A car il est au-dessus : c'est le premier enfant du nœud A (d'indice 0).
- Le nœud D est enfant du nœud B.

Si le nœud A est déplacé de 10 px (pixels) vers la droite, les nœuds B,C et D le seront aussi.
Si le nœud B est déplacé de 15 px vers le haut, le nœud D le sera aussi. Mais les nœuds A et C resteront inchangés : c'est l'hérédité.

1.2 Scènes :

Un arbre de nœuds est appelé une scène. Une scène peut être modifiée et enregistrée pour ensuite être utilisée comme un nœud à par entière. On pourra l'utiliser dans une autre scène (un autre arbre) par exemple, et en autant d'exemplaires que nécessaire. Un scène est donc une structure de base permettant la création d'un objet dans notre projet.

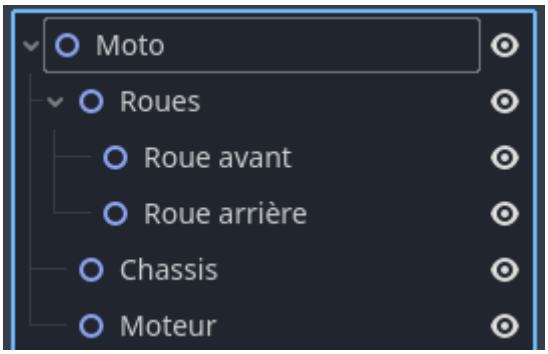


Figure 1.2.1 : Exemple d'une scène

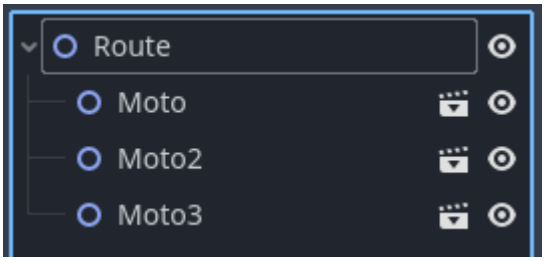


Figure 1.2.2 : Autre exemple de scène

Figure 1.2.1 :

- Ici nous avons une scène 'Moto', contenant des nœuds Roues, un nœud 'Chassis' et un nœud 'Moteur'.

Figure 1.2.2 :

- Cette scène est utilisée à trois reprises dans la scène ‘Route’.
- Chaque nœud ‘Moto’ de la scène ‘Route’ est un instance à par entière de la scène ‘Moto’ ; elle évolue indépendamment des autres.

En conclusion, c’est de la programmation orienté objet.

1.3 Scripts :

Chaque nœud peut être relié à un script : un code en GDScript permettant de contrôler le comportement de ce nœud. Nous pouvons y créer de nouveaux attributs ou modifier des attributs déjà existants : comme, par exemple, ‘position’ qui contient les coordonnées du nœud.

Grâce à la méthode _process(delta), nous pouvons exécuter du script à chaque image du jeu (60ips équivaut donc à 60 exécution par seconde). Le paramètre delta contient le temps écoulés depuis le dernier appel de cette méthode (en secondes).

1.4 Quelques nœuds et propriétés utilisés :

1.4.1 Métadonnées :

Une métadonnée est une valeur quelconque qui peut être stockées dans un nœud quelconque. A la différence d’un attribut personnalisé (crée par le développeur) d’un nœud dont, les métadonnées n’ont pas besoin de script pour exister.

1.4.2 Path2D & PathFoloow2D :

Path2D est un nœud qui contient un chemin à suivre, c’est à dire des points qu’il relie par des lignes (courbes ou droites).

PathFollow2D est un nœud, impérativement enfant d’un Path2D, qui prend le chemin de son parent (le Path2D) et le suit avec une barre de progression.

1.4.3 Area2D & ColisionShape2D :

Area2D est un nœud qui détecte la présence d’objet sur lui, que ce soit un autre nœud ou le curseur de la souris. Il a impérativement besoin d’une ColisionShape2D en nœud enfant pour fonctionner.

ColsionShape2D est nœud qui donne une forme géométrique, cette dernière peut être utilisée pour détecter des collisions, des cliques etc.

2 Game design

Ce chapitre traite de nos décisions sur le contenue et le fonctionnement du jeu.

Dans ‘Chef de Gare’, le joueur incarne un responsable ferroviaire chargé du bon fonctionnement de sa gare. Il doit dirigé les trains vers les bonnes destinations sans faire de collisions entre eux.

2.1 Aspects généraux du jeu :

2.1.1 But du jeu et score :

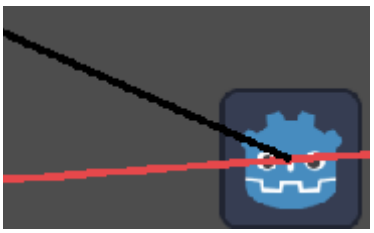
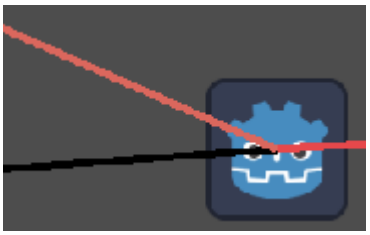
Chaque train a une destination et un quai bien spécifiques. A l’entrée de la gare le joueur doit envoyer le train sur le bon quai, puis, après un certain délai, ensuite le renvoyer vers sa bonne destination. Sans quoi une pénalité lui est infligée. Afin de l’aider dans l’orientation, un jeu de couleur permet de mieux visualiser la destination et le quai de chaque trains.

La performance du joueur est jugée grâce à un score : ce dernier augmente à la disparition de chaque train.
Si le joueur se trompe de quai et/ou de destination, une pénalité sur le score est appliquée.
Si le joueur utilise l’option d’arrêt d’un train (cf. 2.1.2) une pénalité sur les score lui est appliquée.

La partie est perdue en cas de collision de trains.
La difficulté réside dans la gestion rapide de plusieurs trains arrivant quasi simultanément. Le jeu est donc orienté vers du dynamisme plus que de la planification.

2.1.2 Actions du joueur :

Afin de pouvoir aiguiller les trains sur les bonnes voies, le joueur doit gérer un ensemble d’aiguillages interconnectés créant ensemble un complexe ferroviaire. Chaque aiguillage de ce complexe peut être changé avec une pression sur un bouton. Une couleur permet au joueur de savoir dans quelle direction est aiguillés l’aiguillage.

Figures 2.1.2.1 et 2.1.2.1.bis : Exemple d' un aiguillage

Figures 2.1.2.1 et 2.1.2.1.bis :

A gauche : Un aiguillage aiguillé sur la voie du bas
A droite : Un aiguillage aiguillé sur la voie du haut, après que l'utilisateur est appuyé sur le bouton (ici en bleu)

Les trains suivent donc le chemin de la voie en couleur (ici en rouge)

De plus, afin de permettre au joueur de gérer une situation périlleuse, ce dernier peux en arrêter un train en cliquant dessus puis le relancer. Mais cette option est pénalisée par un retrait de point sur le score.

2.1.3 Vue de l'utilisateur :

Le jeu se joue en vue en plan (ou vue du dessus), sans aucune perspective. Cela permet à l'utilisateur de visualiser en direct l'état de l'entièreté du complexe ferroviaire de la gare, de prédire plus facilement les déplacements des trains et de pouvoir réagir rapidement en conséquence grâce à un accès simple aux boutons d'aiguillages et aux trains (simple mouvement de la souris).

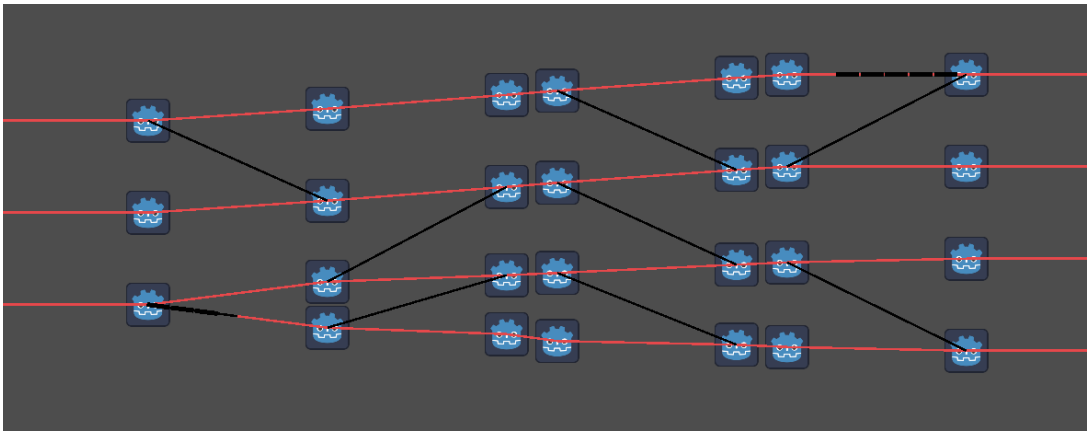


Figure 2.1.3.1 : Exemple schématique d'un complexe ferroviaire (en vue du dessus)

Figure 2.1.3.1 :

En bleu, les aiguillages
En rouge, les voies actives (les voies aiguillées)
En noire, les voies passives, les trains ne peuvent les emprunter
Rectangles noires, les trains (un rectangle correspond à un wagon)

2.2 Aspects spécifiques du jeu :

2.2.1 Apparition des trains :

Afin de faire un jeu jouable, des trains doivent apparaître en boucle pour remplacer les anciens.
Pour ce faire, deux procédés s'offrent à nous :

- Les générer de manière aléatoire. Il faut donc trouver un algorithme qui va résoudre ce problème sans créer situation absurde (un train apparaît sur la même voie qu'un autre mais dans le sens opposé etc.)
- Les générer avec des patterns pré-générer qui s'enchaînent, peut être plus simple mais est complexe à mettre en place si on considère la possibilité d'arrêter un train

Nous avons choisi l'option du générateur aléatoire. Le moment d'apparition, le quai et la destination sont donc déterminée de manière pseudo-aléatoire par un algorithme défini.

L'arrivée de chaque train est annoncée légèrement en avance, pour prévenir le joueur.

2.2.2 Type de carte :

Le type de carte est une gare terminus (ou gare cul-de-sac), à la manière de la gare de Tours Centre. Les trains arrivent de la droite et se dirigent vers un quai à gauche. Passé un certain délai, il reparte pour sortir de la gare à droite.
En résumé les quais sont à gauche, les arrivées et départs des trains sont à droites.

Ce type de gare permet de diminuer la taille de la carte tout en gardant une certaine variété de jeu car des train circulent dans les deux sens sur les mêmes rails. Une gare classique comme celle de Saint Pierre des Corps auraient pris bien plus d'espace sans forcément rendre le jeu plus intéressant.

2.2.3 Difficulté :

La difficulté n'a pas encore été implémentée au 11 mai 2025.

La difficulté est garantie par la présence de niveaux de difficultés, changeant :

- la taille de la carte (quantité de quai et de voies),
- la vitesse des trains,
- les différents temps des morts des trains (au quai et en arrêt)
- la quantité de trains
- la probabilités d'apparition d'un train
- le vitesse de jeu

La vitesse de jeu est un nombre, augmentant au cours d'une partie, qui celle-ci plus compliquée (vitesse, temps morts etc.)

3 Développement du jeu

Cette section traite des techniques utilisées pour crée notre jeu :

3.1 Aspects généraux du jeu :

3.1.1 Création de la carte :

‘Chef de Gare’ utilise un graphe spéciale pour simuler son complexe ferroviaire. Ce graphe est composé de points reliés entre eux, ce sont eux qui crée les voie pour les trains et permettent de créer des aiguillages.
Le graphe est divisé en couche afin de simplifier la gestion des itinéraires des trains. Chaque couche correspond à un alignement verticale de points de ce graphe.

Chaque point du graphe appartiens à une couche n, il est ainsi lié à un point de couche n+1 et un point de couche n-1 ; les adresses d’accès à ces points sont stockés dans la métadonnée ‘Connections’ (cf. 0.4.1) du point n. Les points en extrémité du complexe ferroviaire ne sont donc liés qu’à un seul autre point du graphe. Cette liaison permet à un train passant par un point du graphe, de connaître le point suivant et donc ses coordonnées.

Dans le cas d’un aiguillage, une métadonnée supplémentaire apparaît. Cette dernière contient les 2 points s’alternant sur cet aiguillage.
En cas d’activation de l’aiguillage, le point se trouvant dans la métadonnée ‘Connections’ et remplacé par celui qui ne s’y trouve pas, permettant ainsi un alternance fluide.

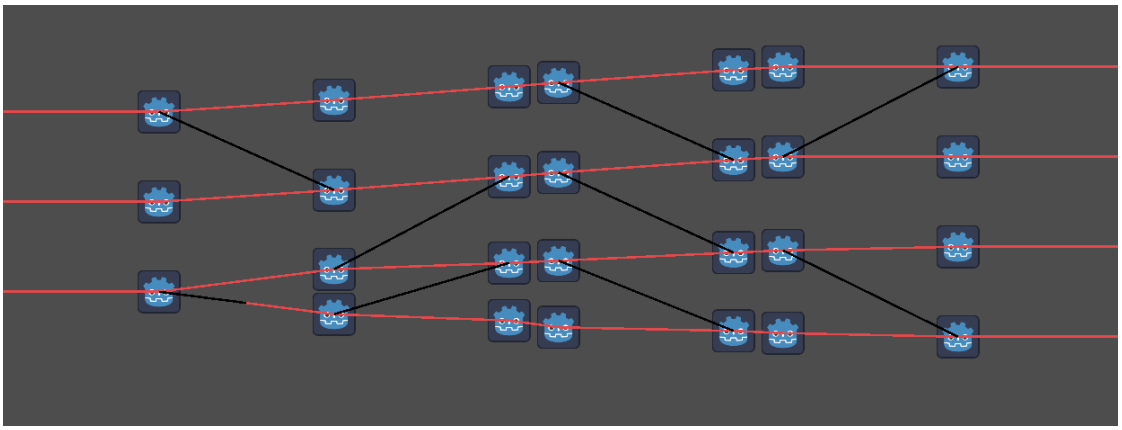


Figure 3.1.1.1 : Exemple d’un carte de jeu

Figure 3.1.1.1 :

- Chaque ligne verticale de carrées bleus est une couche du graphe.
- Chaque lignes est une connections entre deux points :
  - les rouges sont les connections actives,
  - les noires sont les connections désactivées.
- Nous pouvons voir que des points d'une couche, ne sont reliés qu'à des points d'une couche suivante ou précédente.
- Les extrémités du graphe se situent volontairement à l'extérieur de la fenêtre d'affichage (droite et gauche).

3.1.2 Chemin d'un train :

Pour connaître son chemin, un train se sert de son origine, un paramètre qui est donnée lors de la création du train. Cette origine contient le point du graphe à l'origine du train. Le train pioche dans la métadonnée 'Connections' du point à l'origine et peut ainsi connaître le point du graphe suivant par le quel il doit passer. Il fait ainsi de suite jusqu'à arriver à la fin du graphe.
Une fois toutes les coordonnées récoltés, le train construit, grâce au nœud Path2D (cf.0.4.2) le chemin qu'il doit prendre.

Dans le cas d'un changement d'un aiguillage, le train recalcule le chemin qu'il doit faire, mais cette fois sont point à l'origine est le dernier point par lequel il est passé.

Pour optimiser le jeu, le train ne recalcule son chemin que si c'est nécessaire, c'est à dire seulement quand un point par le quel il est censé passer a un changement d'aiguillage.

Par exemple si le joueur change l'aiguillage d'un point par lequel, pour l'instant, aucun train ne passe ; aucun train ne recalcule sont chemin.

Une fois ce chemin construit, le train peut le suivre grâce à un pathFollow2D (cf.0.4.2) : on nœud qui permet de localiser un objet grâce à un chemin (Path2D) et un progression sur ce chemin.

3.1.3 Détection de collisions et cliques du joueur :

Pour détecter des collisions entre deux trains, 'Chef de Gare' utilise des nœuds Area2D (cf.0.4.3). Ces derniers permettent de détecter l'entrée et la sortie d'objet dans une zone spécifique.
Ainsi quand deux Area2D de deux trains différents de chevauchent, ces derniers s'en rendent compte et cela interrompt la partie.

Les cliques du joueurs sur des aiguillages utilisent aussi des Area2D

3.2 Aspects spécifiques du jeu :

3.2.1 Génération aléatoire de train :

Pour générer des trains de manière aléatoire, ‘Chef de Gare’ suit un algorithme bien défini.

1. Il vérifie si le nombre de train est au maximum (il y a une limite finie de train simultanée correspondant au nombre de voie)
2. Il vérifie si une origine est disponible (aucun train ne doit obstruer)
3. Il vérifie si un quai est disponible (aucun train ne doit obstruer)
4. Il vérifie si une destination est disponible (aucun train ne doit obstruer)
5. Si ces conditions son réunis, il lance une fonction probabiliste qui a 1 chance sur n de générer le train.

Sinon l’algorithme est rappelée à l’image suivante

Si un train est généré, il y a un temps morts de quelques secondes avant le prochain train.

4 Projet fini

Ce chapitre traite de notre vision d'un projet fini, ce que l'on aurait aimé atteindre dans l'absolue.

Le projet fini est un jeu où nous avons le choix entre 3 difficultés : facile, moyen et difficile. Chacune étant un niveau à part entière, changeant la taille de la carte de jeu (Quantité de quais, de voies, d'aiguillages), l'intensité de jeu (vitesse des trains, fréquence d'apparition..) etc.
Un tableau d'affichage indique l'arrivé prochaine des trains, la voie concernée ainsi que le quai. Puis leurs voie par lequel il repart.
Des couleurs mettent en liens les trains et les quais ou sorties vers lesquels ils se dirigent, permettant au joueur de visualiser rapidement la destination de chaque train.
Pour amener les trains au bon quai, le joueur peut cliquer sur des aiguillages afin de changer les voies aiguillées. Les voies aiguillées sont indiquées en rouge.
Le joueur peut, s'il le faut, cliquer sur un train afin de l'arrêter jusqu'au prochain clique, mais sous peine de recevoir une pénalité sous la forme de perte de points. Des points sont aussi retirés si le joueur se trompe de quai ou de voie de sortie. Le joueur perd la partie si 2 trains rentrent en collision.

L'algorithme de résolution reste absent de nos attentes car trop complexe à mettre en œuvre et par manque de temps.

5 Conclusion

Au 11 mai 2025, une bonne partie des objectifs fixés on été atteint : le jeu est jouable et les trains apparaissent de manière aléatoire et infinie, néanmoins des améliorations sont toujours possible, notamment et principalement au niveau des graphismes, et la réalisation des objectifs de difficulté (différents niveaux et difficulté croissante au cours d'une partie) est toujours possible.

L'algorithme de résolution automatique ne pourra, lui, pas être réalisé. C'est un objectif difficile et Godot n'aide pas à sa réalisation.

Ça a été une expérience enrichissante qui nous en a appris d'avantage sur la réalisation d'un jeu vidéo, et plus globalement, sur la gestion d'un projet informatique. Notamment la difficulté et l'organisation nécessaire pour la surmonter. Mais aussi la quantité de travail demandé
Ce fut néanmoins un projet très plaisant à faire, et nous espérons pouvoir l'améliorer afin de pouvoir atteindre la quasi-totalité de nos objectifs.

Même si le jeu final n'est pas très amusant à jouer, nous sommes fière de ce que nous avons accomplit et nous espérons que vous apprécierez ce jeu autant que nous avons appréciez le créer.

6 Bibliographie

6.1 Playlist pour découvrir et à apprendre à utiliser le moteur Godot :
https://www.youtube.com/watch?v=q7wlSvt0JIc&list=PL4cUxeGkcC9iHCXBpxbdsO-ByZ55Ez4bgF

6.2 Documentation Godot :
https://docs.godotengine.org/en/stable/

6.2.1 Documentation Path2D & PathFollow2D:
https://docs.godotengine.org/en/stable/classes/class_path2d.html
https://docs.godotengine.org/en/stable/classes/class_pathfollow2d.html

6.2.2 Documentation Area2D & CollisionShape2D:
https://docs.godotengine.org/en/stable/classes/class_area2d.html
https://docs.godotengine.org/en/stable/classes/class_collisionshape2d.html

7 Comptes rendus hebdomadaires

Compte rendu du 22/01/2025

Bonsoir,
Je suis Hamdouni Elias du groupe 2 de PEIP1 et je suis en binôme avec Ivan, aussi du groupe 2. Lors de la première et deuxième séance de projet, mon binôme et moi avons établit notre cahier des charges pour le jeu, nous nous sommes mis d'accords sur le principe basique du jeu ainsi que sur le gameplay. Nous avons trier en 3 les fonctionnalités que nous voudrions développer en fonction de leurs priorité et nécessité. Néanmoins nous hésitons encore sur le modèle de gare que nous devrions faire : terminus comme Tours ou de passage comme St-Pierre-des-Corps. Nous avons aussi réfléchis un peu en avance au condition que devra vérifier l'algorithme de résolution pour fonctionner correctement ainsi que l'algorithme de création des trains. Mon camarade vous a demandé un rendez-vous afin que nous puissions faire valider nos idées, avant de passer à la programmation.
Cordialement,
Elias Hamdouni

Compte rendu du 31/01/2025

Bonjour,
je suis Hamdouni Elias, élève de PEIP1 G2, spécialité math et mon collègue Ivan est de la même classe. Voici mon rapport sur l'avancement de notre projet sur le jeu chef de gare : nous avons eu un rendez-vous avec vous sur le cahier des charges que vous avez fait valider et nous nous sommes renseignés sur comment utiliser le langage GODOT qui est le langage que nous utilisons pour notre jeu.
Cordialement,
Hamdouni Elias

Compte rendu du 09/02/2025

Bonjour,
Je suis Elias Hamdouni, élève de PEIP1 M, du G2, avec comme binôme Ivan. Voici le rapport hebdomadaire sur l'avancement du projet. Nous avons commencer la programmation avec : -pouvoir faire suivre le chemin au train et modifier le chemin (en cours) -changer la valeur envoyé d'un aiguillage entre 2 valeurs possibles quand on lui clique dessus (finit) pour dans le futur modifier le chemin pour le train A cause de partiel nous n'avons pas pu beaucoup travailler, mais ceux-ci étant finit pour quelques semaines, nous allons pouvoir correctement travailler. Nous aimerions prendre rendez-vous avec vous la semaine de la rentré, plus spécifiquement le lundi 24/02/2025 de 10h30 à 12h30 pendant notre séance de projet, est-ce possible pour vous ?
Cordialement,
Elias Hamdouni

Compte rendu du 23/02/2025

Bonjour,
Je suis Elias Hamdouni, élève de PEIP1 Math, G2, avec mon camarade Ivan du même groupe. Voici mon rapport de la semaine sur l'avancement du projet chef de gare : -début de l'UI avec possibilité d'afficher des horaires, quai... -déplacement des trains sur les voies invisible, et redirection possible sans lien avec les aiguillages Je vous souhaite une bonne journée
Cordialement,
Elias Hamdouni

Compte rendu_du_03/03/2025

Bonjour,
Je suis Elias Hamdouni de PEIP1 Math, du G2, avec mon camarade Ivan du même groupe. Voici l'avancement du projet chef de gare. A cause de nos examens très proche et de mon camarade qui est tombé malade, nous n'avons pas pus faire beaucoup de progrès, nous avons commencé la mise en place d'un signal afin de pouvoir raccorder les aiguillages au déplacements des chemins des trains.
Cordialement,
Elias Hamdouni

Compte rendu_du_10/03/2025

Bonjour,
Je suis Elias Hamdouni, élève de PEIP1 Math, du G2 avec comme binôme Yvan, du même groupe. Voici nos avancées pour le projet chef de gare : Nous sommes actuellement entrain de réfléchir à comment vérifier que le train ne soit pas encore passé au moment de l'aiguillage afin de pouvoir modifier sa trajectoire si l'aiguillage est activé. À cause des révisions pour nos évaluations, ainsi que mon binôme qui ne se sentait pas bien, nous n'avons pas beaucoup progressé.
Cordialement, Elias
Hamdouni

Compte rendu_du_17/03/2025

Bonjour,
Je suis Elias Hamdouni, élève de PEIP1 Math du G2 et mon binôme est Ivan du même groupe. Voici mon rapport sur l'avancement du projet : Il n'y a pas beaucoup de progrès depuis la dernière fois suite à la difficulté de la tâche : nous essayons toujours de modifier le chemin que peut prendre un train en prenant en compte quels aiguillages ont été activé.
Cordialement,
Elias Hamdouni

Compte rendu_du_23/03/2025

Bonjour,
Je suis Elias Hamdouni, élève de PEIP1 MATH, du G2, avec comme binôme Yvan du même groupe. Voici mon rapport sur l'avancé du projet : -Début de la création des modèles de trains, aiguillages et rails. -Un train peut désormais modifier sa trajectoire sur les rails si nous cliquons sur les aiguillages tout en prenant en compte si le train a déjà dépassé l'aiguillage et donc si cela l'affecte ou non. -Début de la création du premier niveau.
Cordialement,
Elias Hamdouni

Compte rendu_du_31/03/2025

Bonjour,
Je suis Elias Hamdouni, élève de PEIP1 MATH, du groupe 2, mon binôme Ivan est dans le même groupe. Voici mon rapport sur l'avancé de notre projet chef de gare : -désormais les aiguillages fonctionnes dans les 2 sens, c'est à dire que si les branches se trouvent à gauche ou à droite, l'aiguillage fonctionne -après quelques secondes passé au bout des rails, le train fera demi tour, et peut aller dans n'importe qu'elle sens -correction de bug Je tiens a m'excuser du retard.
Cordialement,
Elias Hamdouni

Compte rendu du 07/04/2025

Bonjour,
Je suis Elias Hamdouni élève de PEIP1 Math du G2 et mon binôme qui est du même groupe est Ivan. Voici mon rapport d'avancé sur le projet chef de gare : -la map est fini -début de la génération aléatoire des trains à plusieurs endroits différents
Cordialement,
Elias Hamdouni

Compte rendu du 14/04/2025

Bonjour,
Je suis Hamdouni Elias, élève de PEIP1 Math du G2 et mon binôme qui est du même groupe s'appelle Ivan. Voici mon rapport sur l'avancé du projet chef de gare : Début de l'apparition de plusieurs train de manière simultané, reste très buggé Un train peut apparaître sur chaque ligne de manière aléatoire
Cordialement,
Elias Hamdouni

Compte rendu du 20/04/2025

Bonjour,
Je suis Hamdouni Elias élève de PEIP1 Math du G2 et mon binôme qui est du même groupe s'appelle Ivan. Je vous informe que depuis la semaine dernière, nous n'avons pas fait de progrès pour le projet chef de gare.
Cordialement,
Hamdouni Elias

Compte rendu du 05/05/2025

Bonjour,
Je suis Hamdouni Elias élève de PEIP1 Math du G2 et mon binôme qui est du même groupe s'appelle Ivan. Je tiens à présenter mes excuses pour le retard du rapport, en effet les examens m'ont totalement fait oublier ce rapport d'avancement pour le projet chef de gare. Du aux examens, nous n'avons pas beaucoup travaillé sur le projet. Néanmoins nous pouvons correctement faire apparaître plusieurs trains aléatoirement et nous avons un compteur de points (bien que ce soit juste une variable et qu'il ne soit pas affiché).
Cordialement,
Elias Hamdouni

‘Chef de Gare’

Résumé

Ce rapport présente en détail les moyens et les différentes étapes utilisées dans la réalisation du projet ‘Chef de Gare’, un projet de jeu. Il traite tout aussi bien du développement de ce dernier, que de sa conception. Mais aussi de l’organisation et la partage des taches derrière ce projet ainsi que le choix de la technologie.

Mots-clés

Développement informatique, programmation, jeu vidéo, Godot

Abstract

This report deals with the way and the different stages behind the conception of the game ‘Chef de Gare’. It explains the game’s conception, it’s development, the team’s organisation and work shearing and the choice of the game engine

Keywords

Game development, programming, video game, Godot

Encadrant académique	Étudiant(e)s
Yannick Kergosien	*Ivan Epov* *Elias Hamdouni*